Infrastructure Network Design Under Budget Constraints

Course Project

Submitted in partial fulfillment of the requirements for

Introduction to Software Programming and Algorithms

in

M.Tech Programme

by

Ankitha G (26894)
Yakshith Maddineni (26080)

Supervisor: **Prof. Nagarajan Natarajan**

Github repository link:

GitHub Repository

Department of CiSTUP

Indian Institute of Science, Bengaluru 560012

Date: 30th April, 2026

Contents

1 Introduction and Problem Statement **5**

1.1 Motivation 5

1.2 Problem Formulation 5

1.3 Research Question 6

1.4 Scope and Datasets 6

2 Methodology **8**

2.1 MST Baseline: Kruskal's Algorithm 8

2.2 Greedy Algorithms 8

2.2.1 Greedy 1: Minimum Cost First (No Cycle Check) 8

2.2.2 Greedy 2: Minimum Cost First (With Cycle Check) 9

2.2.3 Greedy 3: Maximum Gain-per-Cost (ρ-Greedy) 9

2.2.4 Unifying View 10

2.3 Exact DP: Knapsack-Style Optimisation 11

2.3.1 Problem Formulation 11

2.3.2 Implementation 11

2.3.3 Scalability Constraint 12

3 Experiments and Results **13**

3.1 Synthetic Graph Experiments 13

3.1.1 Graph Design and Motivation 13

3.1.2 Algorithm Comparison at $B = 14$ 14

3.1.3 Budget Sweep: Synthetic Graph 15

3.2 Malleshwaram Road Network Experiments 17

3.2.1 Graph Setup 17

3.2.2 Algorithm Comparison at 50% MST Budget 17

3.2.3 Budget Sweep: Malleshwaram 18

3.3 Timing Summary 19

4 Analysis and Discussion **20**

4.1 Answering the Core Research Question 20

4.2 Structural Explanation: Why Greedy 3 Works 20
4.3 The Role of the MST 21
4.4 Scalability 21

5 Limitations and Future Work **22**
5.1 Limitations 22
5.1.1 Cost Model: Length Only 22
5.1.2 Demand Blindness 22
5.1.3 Undirected Graph 22
5.1.4 Homogeneous Budget 22
5.1.5 Static Network 23
5.1.6 No Ground Truth at Scale 23
5.1.7 Topological Connectivity Only 23
5.2 Future Work 23
5.2.1 Richer Cost Models 23
5.2.2 Population-Weighted Objective 23
5.2.3 Directed Graph Extension 24
5.2.4 Theoretical Approximation Bounds 24
5.2.5 Hierarchical Budget Allocation 24
5.2.6 LP Upper Bounds at Scale 24
5.2.7 Multi-Phase Planning 24

6 Conclusion **25**

List of Tables

2.1 Comparison of greedy selection metrics 10

3.1 Algorithm performance on the adversarial synthetic graph ($B = 14$, max pairs = 120) 14
3.2 Budget sweep results on the adversarial synthetic graph (connected pairs) 16
3.3 Algorithm performance on the Malleshwaram graph at 50% MST budget (Rs. 46,938) 17
3.4 Runtime comparison on the synthetic graph at $B = 14$ 19

4.1 Scalability comparison across the two experimental graphs 21

List of Figures

3.1 The adversarial synthetic graph: 16 nodes, 57 edges. Left cluster (nodes 0–7) and right cluster (nodes 8–15) are each fully connected internally with cost-1 edges. A single bridge (edge 0–8) connects the two clusters at cost 2. The spatial separation visually reinforces the connectivity tradeoff that the experiment is designed to probe. 14
3.2 Visual comparison of all five algorithms on the adversarial synthetic graph at $B = 14$. Red edges are selected; grey edges are not. MST achieves full connectivity but requires Rs. 16, exceeding the budget. Greedy 1 forms dense, cycle-heavy subgraphs with poor cross-cluster coverage. Greedy 2 fully connects each cluster separately but never bridges the gap. Greedy 3 and DP select the bridge and distribute remaining budget efficiently, both reaching 91 pairs. 15

3.3 Budget sweep on the adversarial synthetic graph. Left: connected pairs achieved by each algorithm at each budget level. Right: each greedy's performance as a percentage of DP optimal. Greedy 3 tracks DP exactly across all budget levels; Greedy 2 recovers only at 35% when the bridge finally becomes affordable; Greedy 1 never fully recovers within the tested range. 16
3.4 Spatial comparison of the three greedy algorithms on the Malleshwaram road network at 50% MST budget (Rs. 46,938). Red edges are selected; blue dots are nodes. Greedy 3 (rightmost) forms larger connected regions by concentrating on high-leverage bridge roads, resulting in roughly 17 times more reachable pairs than Greedy 1 or Greedy 2. 18
3.5 Budget sweep on the Malleshwaram road network. Left: absolute connected pairs. Right: Greedy 1 and Greedy 2 as a percentage of Greedy 3, which serves as the practical upper bound in the absence of an exact solution. 18

Chapter 1

Introduction and Problem Statement

1.1 Motivation

Designing road infrastructure under financial constraints is one of the most consequential decisions in urban planning. A city planner must decide which roads to build given a fixed budget, with the goal of connecting as many people and localities as possible. Spending too little in a region leaves communities isolated; concentrating the entire budget in one area starves the rest of the network. Finding a principled, computationally efficient way to allocate a limited budget across a road network is therefore both practically urgent and algorithmically interesting.

This project approaches that question through the lens of combinatorial optimisation. Cities are modelled as graph nodes, possible roads as weighted edges where weight represents construction cost, and the question becomes: given a budget B, which subset of edges, when built, maximises the number of pairs of nodes that can reach each other?

The broader motivation goes beyond road networks. The same structure appears whenever resources must be allocated to maximise reachability in a network, whether in telecommunications, supply chains, or last-mile connectivity. The analysis here focuses on road infrastructure, but the algorithmic conclusions carry over.

1.2 Problem Formulation

Let $G = (V, E)$ be an undirected graph where V is the set of nodes (intersections or localities) and E is the set of candidate edges (possible roads). Each edge $e \in E$ has an associated cost $c(e) \in \mathbb{R}^+$, and there is a fixed budget $B > 0$.

The objective is to select a subset $X \subseteq E$ of edges to build such that the total number of reachable pairs of nodes is maximised:

$$\max_{X \subseteq E} f(X) = \sum_{C \in \mathcal{C}(X)} \binom{|C|}{2} \tag{1.1}$$

where $\mathcal{C}(X)$ is the set of connected components of the subgraph (V, X), and $\binom{|C|}{2} = \frac{|C|(|C|-1)}{2}$ counts unordered reachable pairs within component C.

This is subject to the budget constraint:

$$\sum_{e \in X} c(e) \leq B \tag{1.2}$$

It is worth noting that $f(X)$ is a **monotone submodular** function. The marginal gain from adding an edge to a larger set of selected edges is at most as large as the gain from adding it to a smaller set. This structural property is what makes greedy algorithms theoretically grounded candidates for approximation.

1.3 Research Question

Rather than simply implementing these algorithms, the central empirical question is: *Is greedy good enough?* If the gap between the best greedy solution and the true optimal is consistently small across different budget levels and graph structures, that constitutes a practical justification for using greedy on large-scale graphs where exact methods are computationally infeasible.

This is not a trivial question. There are three meaningfully different greedy strategies one could apply, and as the experiments show, they do not perform equally. Part of the contribution of this project is identifying which greedy is good enough and understanding, at an intuitive level, why.

1.4 Scope and Datasets

Two types of graphs are used. The first is an **adversarial synthetic graph**, constructed deliberately to expose the failure modes of each greedy algorithm. It consists of 16 nodes and 57 edges, structured as two dense clusters of 8 nodes each (all internal edges costing 1) connected by a single bridge edge of cost 2. The budget is set to Rs. 14, which is enough to span one full cluster internally plus the bridge, but not enough to span both clusters without careful prioritisation. This design makes the tradeoff between local and global connectivity analytically transparent.

The second is a **real-world road network** of the Malleshwaram neighbourhood in Bengaluru, extracted from OpenStreetMap using OSMnx. This graph contains 2,392 nodes and 3,581 edges, with edge costs set to road length in metres (Rs. 1 per metre). At

this scale, exact methods are computationally infeasible, so the greedy algorithms are compared against each other, with Greedy 3 serving as the practical upper bound.

Chapter 2

Methodology

2.1 MST Baseline: Kruskal's Algorithm

The Minimum Spanning Tree (MST) serves as a structural reference rather than a budget-constrained algorithm. Kruskal's algorithm selects the globally cheapest set of $|V| - 1$ edges that connects all nodes without cycles, ignoring the budget entirely. Its role is twofold: it shows the minimum cost required to achieve full connectivity, and it provides an upper bound on the objective.

The MST was computed using NetworkX's Kruskal implementation with edge weights set to the cost attribute. On the adversarial synthetic graph, the MST cost is Rs. 16, which exceeds the budget of Rs. 14, confirming that the constraint is genuinely tight and full connectivity is not freely achievable within budget.

Time complexity: $O(|E| \log |E|)$. **Space complexity:** $O(|V| + |E|)$.

2.2 Greedy Algorithms

All three greedy algorithms share the same iterative structure: at each step, select one edge according to a criterion, add it to the solution if it fits within the remaining budget and does not form a cycle (for Greedy 2 and Greedy 3), and repeat until no affordable edge remains. The difference lies entirely in the selection criterion.

2.2.1 Greedy 1: Minimum Cost First (No Cycle Check)

Selection rule: At each step, pick the cheapest unselected edge that fits within the remaining budget, with no restriction on cycles.

$$e^* = \arg \min_{\substack{e \in E \setminus X \\ c(e) \leq B - \sum_{e' \in X} c(e')}} c(e) \tag{2.1}$$

This is the simplest possible greedy. Its fundamental flaw is that it permits cycles: it

can spend the entire budget adding redundant edges within an already-connected cluster, never reaching an edge that would bridge it to another component. On the adversarial synthetic graph, with 56 cost-1 intra-cluster edges available, Greedy 1 fills its budget of Rs. 14 entirely with intra-cluster edges, forms multiple cycles, and achieves only 28 connected pairs out of a possible 120.

Time complexity: $O(|E| \log |E|)$ for sorting; $O(|E|)$ for the scan. **Space complexity:** $O(|E|)$.

2.2.2 Greedy 2: Minimum Cost First (With Cycle Check)

Selection rule: Same as Greedy 1, but edges that would form a cycle are rejected using a Union-Find (disjoint set union) data structure.

$$e^* = \arg \min_{\substack{e=(u,v)\in E\setminus X \\ \text{find}(u)\neq\text{find}(v) \\ c(e)\leq B-\sum_{e'\in X} c(e')}} c(e) \tag{2.2}$$

By enforcing acyclicity, Greedy 2 guarantees that every selected edge contributes new connectivity. However, it still sorts by cost alone and does not account for how much connectivity each edge adds. On the adversarial graph, all 56 intra-cluster edges cost Rs. 1, while the bridge costs Rs. 2. Greedy 2 sees 56 cheaper edges and greedily picks them first. After spending Rs. 14 on intra-cluster edges, the budget is exhausted and the bridge is never selected. Both clusters end up fully connected internally but completely isolated from each other, yielding 56 connected pairs. This is a concrete illustration of why cost-based sorting alone is insufficient: the algorithm builds two well-connected sub-networks and misses the one edge that would unite them.

Time complexity: $O(|E| \log |E|)$ for sorting; $O(|E| \cdot \alpha(|V|))$ for Union-Find operations, where α is the inverse Ackermann function (effectively constant in practice). **Space complexity:** $O(|V| + |E|)$.

2.2.3 Greedy 3: Maximum Gain-per-Cost (ρ-Greedy)

Selection rule: At each step, select the edge with the highest efficiency ratio ρ, defined as marginal gain in connected pairs per unit cost.

For edge $e = (u, v)$ connecting components C_u and C_v in the current forest X, the marginal gain is:

$$\Delta(e \mid X) = |C_u| \cdot |C_v| \tag{2.3}$$

and the efficiency ratio is:

$$\rho(e \mid X) = \frac{\Delta(e \mid X)}{c(e)} \tag{2.4}$$

The selection rule is:

$$e^* = \arg \max_{\substack{e=(u,v)\in E\setminus X \\ \text{find}(u)\neq\text{find}(v) \\ c(e)\leq B-\sum_{e'\in X} c(e')}} \rho(e \mid X) \tag{2.5}$$

Importantly, ρ is **recomputed after every edge selection**, because merging two components changes their sizes and therefore the marginal gain of all subsequent candidate edges. On the adversarial graph, the bridge (0, 8) has $\rho = (8 \times 8)/2 = 32$ at the start, while the best intra-cluster edge has $\rho = (1 \times 7)/1 = 7$. Greedy 3 picks the bridge first, immediately connecting all 16 nodes into one potential component, then fills the remaining Rs. 12 of budget with intra-cluster edges to maximise pairs within each cluster, reaching 91 connected pairs. This matches the DP optimal exactly.

The acknowledged limitation of Greedy 3 is that it is myopic: the locally best ρ at each step does not theoretically guarantee global optimality. In practice, however, it performs remarkably well, as the experiments demonstrate.

Time complexity: $O(|E|^2)$ in the worst case, since each of up to $O(|E|)$ rounds scans all remaining edges. **Space complexity:** $O(|V| + |E|)$.

2.2.4 Unifying View

All three greedy algorithms target the same optimisation problem (Equations 1.1 and 1.2) but differ in how they approximate the objective at each step:

Table 2.1: Comparison of greedy selection metrics

Algorithm	**Selection Metric**	**Cycle Check**	**Accounts for Gain?**
Greedy 1	$-c(e)$	No	No
Greedy 2	$-c(e)$	Yes (Union-Find)	No
Greedy 3	$\Delta(e \mid X)/c(e)$	Yes (Union-Find)	Yes

2.3 Exact DP: Knapsack-Style Optimisation

2.3.1 Problem Formulation

The DP formulation finds the provably optimal subset of edges. The decision space is $X \subseteq E$ and the full constraint set is:

$$\max_{X \subseteq E} \; f(X)$$

$$\text{subject to:} \quad \sum_{e \in X} c(e) \leq B \tag{2.6}$$

$$(V(X), X) \text{ is connected} \tag{2.7}$$

$$|X| = |V(X)| - 1 \tag{2.8}$$

Constraints (2.7) and (2.8) together enforce that the selected edges form a single spanning tree over the nodes they touch. In graph theory, these two conditions are exactly equivalent to stating that X forms a tree on $V(X)$: a connected, acyclic subgraph. This rules out both cycles and isolated fragments, ensuring every rupee spent on an edge contributes genuine connectivity.

2.3.2 Implementation

The DP is implemented as a standard 0/1 knapsack over the edge set. The state $dp[b]$ stores the best (connected-pairs count, edge set) achievable within budget b. For each edge in sequence, budget levels are iterated in reverse order to enforce the 0/1 property. The objective $f(\cdot)$ is evaluated via connected-component discovery on the induced subgraph for each candidate state.

Time complexity: $O(|E| \cdot B \cdot (V + E))$, where the $(V + E)$ is the time to compute the count of connected pairs. This is NP-hard in general, and the runtime grows prohibitively with graph size.

Space complexity: $O(B \cdot |E|)$ for the DP table storing edge masks.

Where E is the edge weight, V is the number of nodes, B is the alloted budget.

2.3.3 Scalability Constraint

On the 16-node adversarial graph with $B = 14$, the DP runs in approximately 9 ms. Extrapolating to the Malleshwaram graph (2,392 nodes, 3,581 edges), the estimated runtime exceeds 74 hours. Even with memoisation, the exponential growth in the number of feasible edge subsets and connectivity evaluations is not fundamentally mitigated, so the approach remains computationally impractical at scale. This is precisely what motivates the greedy analysis: demonstrating near-optimality on tractable instances gives empirical confidence in greedy performance on larger instances where no ground truth is computable.

Chapter 3

Experiments and Results

3.1 Synthetic Graph Experiments

3.1.1 Graph Design and Motivation

Unlike a randomly generated graph, the synthetic graph used here is **adversarially constructed** to expose the failure modes of each greedy algorithm. The graph has 16 nodes and 57 edges: two fully connected clusters of 8 nodes each, with all intra-cluster edges having cost 1, connected by a single inter-cluster bridge edge of cost 2. The budget is set to Rs. 14.

The design exploits a fundamental tension in the problem. One cluster can be internally spanned with 7 edges at a total cost of Rs. 7. The bridge costs Rs. 2. That leaves Rs. 5 for the other cluster. An algorithm that does not account for the connectivity gain of the bridge will fill the budget with cheap intra-cluster edges and never cross over. The adversarial graph turns this abstract failure mode into a concrete, measurable outcome.

The MST requires Rs. 16 (7 left-cluster edges + 7 right-cluster edges + 1 bridge), which is above the budget of Rs. 14. Full connectivity is therefore genuinely out of reach; the question is how efficiently each algorithm uses the available budget.

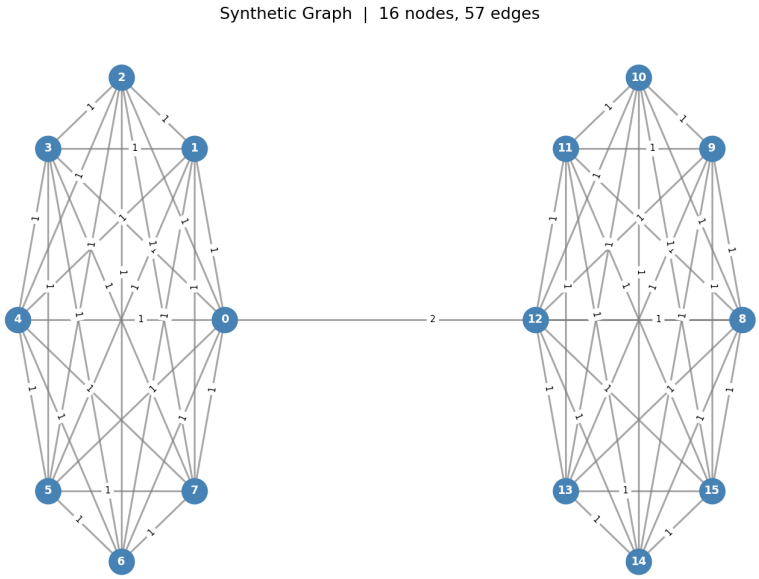

Figure 3.1: The adversarial synthetic graph: 16 nodes, 57 edges. Left cluster (nodes 0–7) and right cluster (nodes 8–15) are each fully connected internally with cost-1 edges. A single bridge (edge 0–8) connects the two clusters at cost 2. The spatial separation visually reinforces the connectivity tradeoff that the experiment is designed to probe.

3.1.2 Algorithm Comparison at $B = 14$

Table 3.1: Algorithm performance on the adversarial synthetic graph ($B = 14$, max pairs $= 120$)

Algorithm	**Pairs**	**Cost (Rs.)**	**Time (ms)**
MST Baseline	120	16	0.467
Greedy 1 (no cycle)	28	14	0.047
Greedy 2 (cycle check)	56	14	0.064
Greedy 3 (max ρ)	91	14	0.232
DP Exact	91	14	9.145

The results in Table 3.1 make the failure modes of each algorithm explicit. The MST costs Rs. 16, confirming the budget is below the full-connectivity threshold and acheives 120 pairs . Greedy 1 spends all Rs. 14 on intra- cluster edges, forms cycles, and achieves only 28 pairs (30.8% of DP optimal). Greedy 2 avoids cycles but still prioritises cheap edges: it picks 14 cost-1 intra-cluster edges, exhausts the budget, and achieves 56 pairs (61.5%) by connecting each cluster internally while leaving the two halves of the network entirely disconnected from each other. Greedy 3 computes $\rho = 32$ for the bridge and $\rho = 7$ for

the best intra-cluster edge, selects the bridge first, and distributes the remaining budget across both clusters, reaching 91 pairs. This matches DP exactly.

The key takeaway is that **cycle detection alone is not sufficient**. Greedy 2 fixes the waste-on-cycles problem of Greedy 1 but introduces a different failure: it prioritises local cheapness over global connectivity gain. Only the gain-per-cost criterion in Greedy 3 correctly identifies which edge creates the most connectivity per rupee spent.

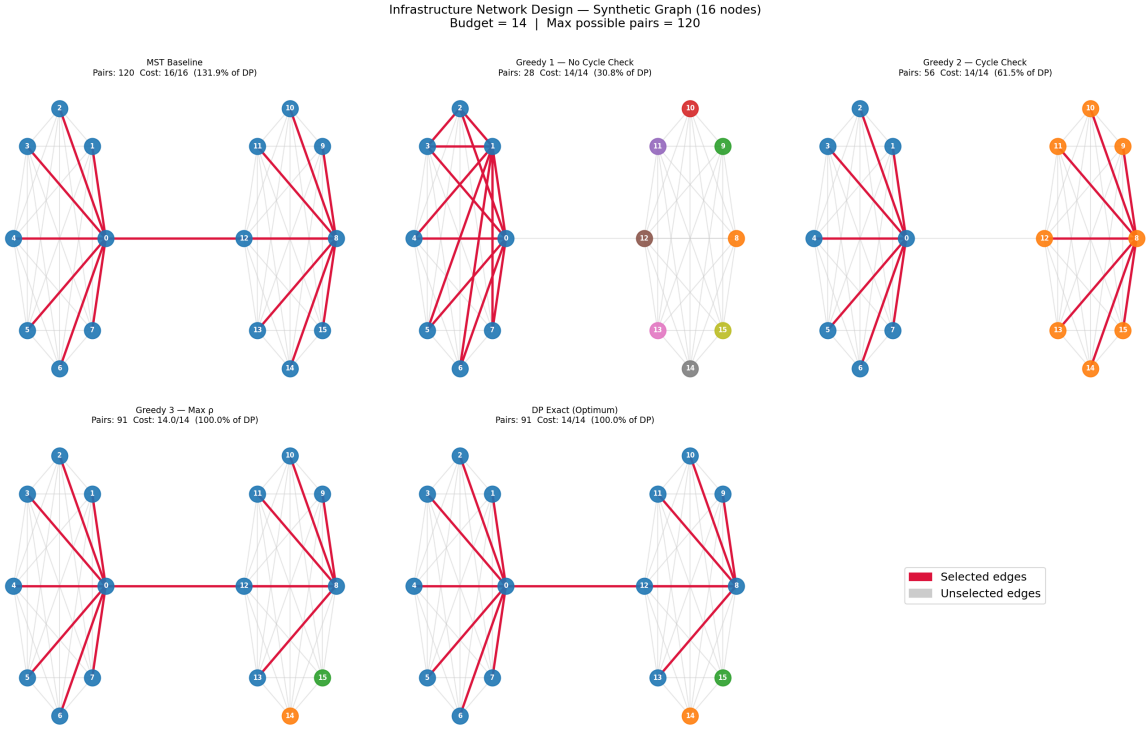

Figure 3.2: Visual comparison of all five algorithms on the adversarial synthetic graph at $B = 14$. Red edges are selected; grey edges are not. MST achieves full connectivity but requires Rs. 16, exceeding the budget. Greedy 1 forms dense, cycle-heavy subgraphs with poor cross-cluster coverage. Greedy 2 fully connects each cluster separately but never bridges the gap. Greedy 3 and DP select the bridge and distribute remaining budget efficiently, both reaching 91 pairs.

3.1.3 Budget Sweep: Synthetic Graph

To understand performance across budget levels, a sweep from 15% to 80% of total edge cost was run. Figure 3.3 shows connected pairs (left) and optimality ratios relative to DP (right).

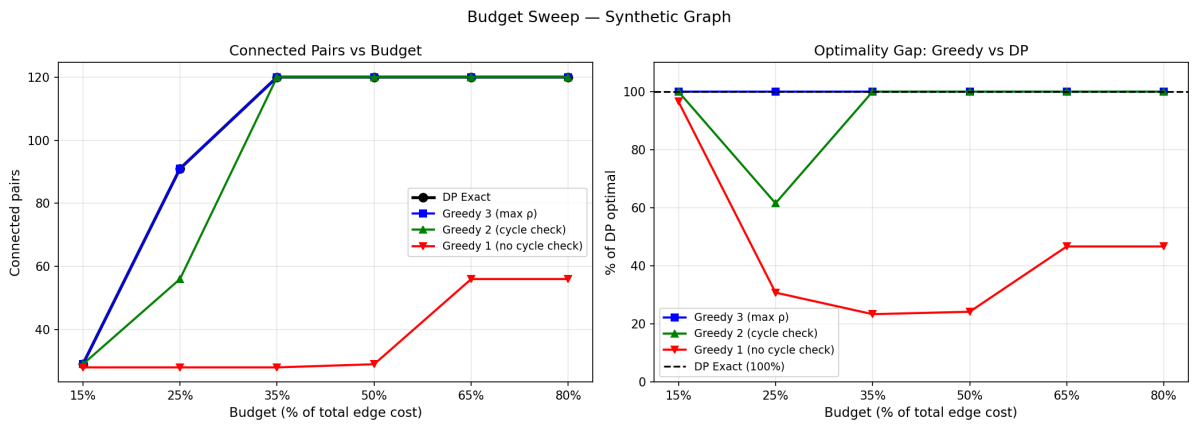

Figure 3.3: Budget sweep on the adversarial synthetic graph. Left: connected pairs achieved by each algorithm at each budget level. Right: each greedy's performance as a percentage of DP optimal. Greedy 3 tracks DP exactly across all budget levels; Greedy 2 recovers only at 35% when the bridge finally becomes affordable; Greedy 1 never fully recovers within the tested range.

Table 3.2: Budget sweep results on the adversarial synthetic graph (connected pairs)

Budget (% of total)	**Greedy 1**	**Greedy 2**	**Greedy 3**	**DP Exact**
15% (Rs. 8)	28	29	29	29
25% (Rs. 14)	28	56	91	91
35% (Rs. 20)	28	120	120	120
50% (Rs. 29)	29	120	120	120
65% (Rs. 37)	56	120	120	120
80% (Rs. 46)	56	120	120	120

The budget sweep in Table 3.2 reinforces the single-budget result. The critical budget level is Rs. 14 (25% of total edge cost), where Greedy 2 achieves 61.5% of optimal and Greedy 3 achieves 100%. Greedy 2 only catches up at Rs. 20 (35%), when the budget is finally large enough to span both intra-cluster trees and still afford the bridge. Greedy 1 recovers partially only at higher budget fractions, where cycle waste has less relative impact. Greedy 3 matches DP at every single budget level tested.

These results are not coincidental artefacts of one particular graph instance; they follow directly from the adversarial structure of the graph, which was designed to make the failure modes of Greedy 1 and Greedy 2 structurally inevitable.

3.2 Malleshwaram Road Network Experiments

3.2.1 Graph Setup

The Malleshwaram subgraph was extracted from OpenStreetMap using OSMnx. After deduplicating parallel edges by keeping the shortest, the graph contains **2,392 nodes** and **3,581 edges**. Edge costs are road lengths in metres, at Rs. 1 per metre. The MST cost is approximately Rs. 58,672 m, i.e., the minimum spend to connect the entire neighbourhood corresponds to building roughly 58.7 km of roads.

Since DP is computationally infeasible at this scale (estimated runtime exceeding 74 hours), Greedy 3 is used as the best available performance reference. Budgets are swept at 20%, 35%, 50%, 65%, and 80% of MST cost.

3.2.2 Algorithm Comparison at 50% MST Budget

At a budget of 50% of MST cost (Rs. 46,938):

Table 3.3: Algorithm performance on the Malleshwaram graph at 50% MST budget (Rs. 46,938)

Algorithm	**Connected Pairs**	**Budget Used (Rs.)**	**Edges Selected**
Greedy 1 (no cycle check)	27,773	46,926	1,728
Greedy 2 (cycle check)	43,691	46,934	1,705
Greedy 3 (max ρ)	759,564	46,934	1,262

The performance gap on the real network is even more pronounced than on the synthetic graph. Greedy 3 achieves 759,564 connected pairs, roughly 17 times more than Greedy 1 (27,773) and approximately 17 times more than Greedy 2 (43,691), while selecting *fewer* edges (1,262 compared to 1,728 and 1,705 respectively). The interpretation is consistent with the synthetic graph findings: Greedy 1 wastes budget on cheap but redundant short roads, Greedy 2 avoids direct redundancy but still selects many low-value intra-cluster roads, and Greedy 3 systematically identifies roads that bridge large disconnected components.

Figure 3.4: Spatial comparison of the three greedy algorithms on the Malleshwaram road network at 50% MST budget (Rs. 46,938). Red edges are selected; blue dots are nodes. Greedy 3 (rightmost) forms larger connected regions by concentrating on high-leverage bridge roads, resulting in roughly 17 times more reachable pairs than Greedy 1 or Greedy 2.

The spatial visualisation in Figure 3.4 makes this difference intuitive. Greedy 1 and Greedy 2 produce many small, scattered components distributed across the map: the budget is being spent on short individual road segments that do not aggregate into large connected zones. Greedy 3 concentrates its selections on roads that link substantial subgraphs together, visible as clusters of red-highlighted roads forming larger contiguous zones, particularly in the north-east section of the Malleshwaram subgraph.

3.2.3 Budget Sweep: Malleshwaram

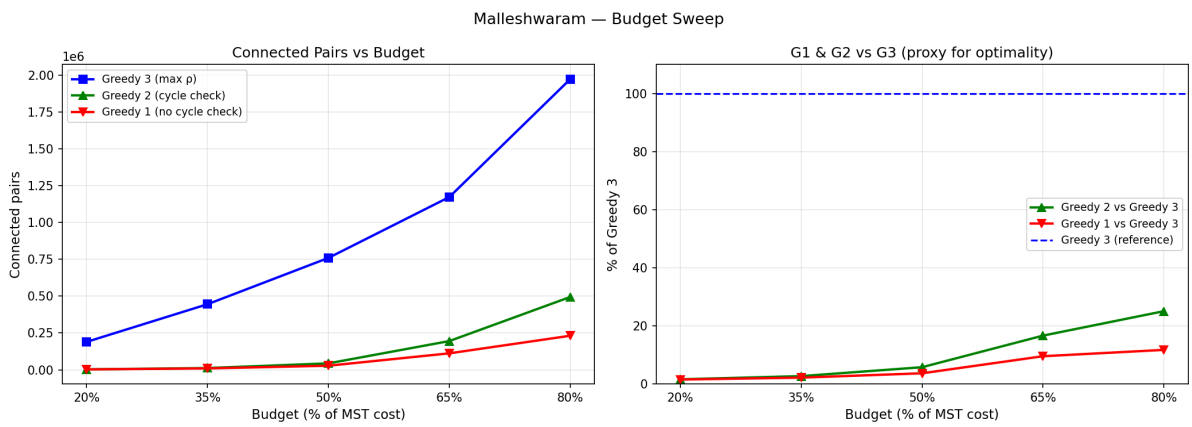

Figure 3.5: Budget sweep on the Malleshwaram road network. Left: absolute connected pairs. Right: Greedy 1 and Greedy 2 as a percentage of Greedy 3, which serves as the practical upper bound in the absence of an exact solution.

As shown in Figure 3.5, Greedy 3's dominance is consistent across every budget level tested. Even at 80% of MST cost, Greedy 1 reaches only about 12% and Greedy 2 about 25% of Greedy 3's connected pairs. The gap narrows somewhat at higher budgets, as more edges become collectively affordable regardless of selection strategy, but the ranking of the three algorithms does not change at any point. The finding from the synthetic graph, that the selection criterion is the dominant factor in performance, extends to the real-world network and holds across the full range of budget fractions.

3.3 Timing Summary

Table 3.4: Runtime comparison on the synthetic graph at $B = 14$

Algorithm	**Time (ms)**	**Speedup vs. DP**
MST Baseline (Kruskal's)	0.467	20×
Greedy 1 (no cycle check)	0.047	195×
Greedy 2 (cycle check)	0.064	143×
Greedy 3 (max ρ)	0.232	39×
DP Exact	9.145	1×

Greedy 3 achieves 100% of DP optimal while running approximately 39 times faster on the synthetic graph. At Malleshwaram's scale, Greedy 3 completes in well under a second while DP is estimated to require over 74 hours. This gap is not a constant factor that better hardware can bridge; it reflects the fundamental NP-hardness of the exact problem. Greedy 3's $O(|E|^2)$ complexity, while not linear, scales comfortably to thousands of edges and can be reduced further with priority queue implementations for even larger networks.

Chapter 4

Analysis and Discussion

4.1 Answering the Core Research Question

The central question was: *Is greedy good enough?* The answer is yes, but only for the right greedy. Greedy 3 is reliably good enough across all settings tested. Greedy 2 is conditionally adequate when cheap edges are uniformly distributed and no single high-cost bridge dominates connectivity gain. Greedy 1 is not good enough in any budget-constrained setting.

On the adversarial synthetic graph at the critical budget level, Greedy 3 matches DP exactly, Greedy 2 achieves 61.5%, and Greedy 1 achieves 30.8%. On Malleshwaram, Greedy 3 outperforms the other two by a factor of roughly 17. Across all budget levels on both graphs, the ranking Greedy 3 $\gg$ Greedy 2 $>$ Greedy 1 is stable and consistent.

4.2 Structural Explanation: Why Greedy 3 Works

The theoretical basis for Greedy 3's strong performance lies in the submodular structure of the objective. The function $f(X) = \sum_C \binom{|C|}{2}$ is monotone submodular. For budget-constrained maximisation of a monotone submodular function, the gain-per-cost greedy is known to achieve an approximation ratio of $(1-1/e) \approx 0.632$ of the optimum in the worst case. The empirical results here substantially exceed this bound, which suggests that real and structured graphs (both random and road networks) have additional properties that the worst-case analysis does not exploit.

The ρ criterion also has a natural economic interpretation. Adding an edge that merges two components of sizes a and b adds $a \cdot b$ new reachable pairs. If this edge costs c, the return per rupee of infrastructure budget is ab/c. An algorithm that maximises this ratio at every step is asking: where does the next rupee of infrastructure spending create the most new connectivity? That is precisely the right question for this problem, and it is what both Greedy 1 and Greedy 2 fail to ask.

4.3 The Role of the MST

An observation from the results is that Greedy 2 and Greedy 3 both recover a spanning tree once the budget exceeds the MST cost. This is structurally guaranteed: both enforce acyclicity, so they can select at most $|V| - 1$ edges, and when the budget is large enough, they eventually construct a full spanning tree. The MST is therefore not just a reference upper bound — it is the natural attractor of any acyclic greedy given sufficient budget. This also explains why the MST cost is a more meaningful budget normalisation baseline than total edge cost: it marks the threshold at which the problem transitions from constrained to unconstrained.

4.4 Scalability

Table 4.1: Scalability comparison across the two experimental graphs

Graph	**Nodes**	**Edges**	**DP Feasible?**	**Greedy 3 Runtime**
Synthetic (adversarial)	16	57	Yes (∼9 ms)	<1 ms
Malleshwaram	2,392	3,581	No (∼74 hrs est.)	<1 s

The scalability difference between exact and greedy approaches is fundamental. DP's runtime grows with both $|E|$ and B, with an additional $O(|V|)$ factor from evaluating $f(\cdot)$ inside the loop. Greedy 3 is $O(|E|^2)$ in the naive implementation but this is manageable up to tens of thousands of edges. The practical conclusion is that for any real-world infrastructure planning problem at meaningful scale, Greedy 3 is the only viable option among the algorithms studied here.

Chapter 5

Limitations and Future Work

5.1 Limitations

5.1.1 Cost Model: Length Only

The most significant simplification is that edge cost equals road length at Rs. 1 per metre. Real road construction costs depend on terrain and elevation, road width and number of lanes, pavement type, land acquisition costs, utility relocation, and labour rates. A more realistic cost function would be multi-attribute and likely non-linear, since cost per kilometre varies substantially between flat urban corridors and constrained or hilly stretches.

5.1.2 Demand Blindness

The objective $f(X) = \sum_C \binom{|C|}{2}$ treats all node pairs as equally valuable. In practice, connectivity between two high-density localities creates far more social and economic value than connectivity between two sparsely used junctions. The current formulation has no mechanism to incorporate population density, travel demand, or economic activity, so the algorithmically optimal solution may not correspond to the most beneficial real-world allocation.

5.1.3 Undirected Graph

The model uses an undirected graph, which ignores one-way streets, turn restrictions, and the generally asymmetric nature of urban road networks. In a city like Bengaluru, where one-ways are common, the undirected assumption may produce edge selections that do not cleanly translate into operational connectivity.

5.1.4 Homogeneous Budget

A single unified budget pool is used, whereas real infrastructure planning involves separate funding streams for arterial roads, collector roads, and local streets, each with different

cost profiles and administrative constraints. A single-pool model misses these structural realities.

5.1.5 Static Network

The OSMnx graph is a point-in-time snapshot. Roads degrade over time, new routes are opened, and land-use changes alter the set of feasible edges. There is no temporal dimension in the current formulation.

5.1.6 No Ground Truth at Scale

On the Malleshwaram graph, DP is infeasible, so Greedy 3 serves as a reference rather than a true optimum. The actual optimality gap of Greedy 3 on this graph is unknown. The justification for Greedy 3 rests on its exact optimality on the synthetic graph and its dominance over other greedy algorithms, not on a direct comparison to a verified optimal solution at scale.

5.1.7 Topological Connectivity Only

The reachable-pairs metric captures whether two nodes are connected at all, but not how well they are connected in terms of travel time, path length, or road quality. Two nodes connected by a long, indirect detour count identically to two nodes with a direct link.

5.2 Future Work

5.2.1 Richer Cost Models

Incorporating multi-attribute cost functions that account for terrain, road width, and land-use type would make the optimisation more practically meaningful. Datasets from BBMP or NHAI could potentially serve as a basis for this.

5.2.2 Population-Weighted Objective

Replacing $f(X)$ with a demand-weighted version, $\tilde{f}(X) = \sum_C \sum_{u,v \in C} p_u \cdot p_v$, where p_u is the population or activity level at node u, would better capture the social value of connectivity improvements.

5.2.3 Directed Graph Extension

Extending to directed graphs with asymmetric edge costs and one-way constraints would produce more realistic solutions for dense urban road networks.

5.2.4 Theoretical Approximation Bounds

Verifying whether the empirical approximation ratio of Greedy 3 consistently exceeds the $(1 - 1/e)$ theoretical bound, and identifying graph-structural properties that drive this behaviour, would connect the empirical findings to the broader literature on submodular function maximisation.

5.2.5 Hierarchical Budget Allocation

Modelling separate budget pools for different road categories and jointly optimising within and across levels would better reflect how real infrastructure budgets are structured and administered.

5.2.6 LP Upper Bounds at Scale

For graphs where DP is infeasible, LP relaxations can serve as upper bounds on the optimal value, enabling a meaningful optimality gap estimate for Greedy 3 even without the exact solution. This would strengthen the empirical justification for Greedy 3 on large graphs.

5.2.7 Multi-Phase Planning

A staged variant of the problem, where a fraction of the budget is allocated per year and early decisions constrain later options, would better reflect how large infrastructure investments are planned and executed in practice.

Chapter 6

Conclusion

This project studied the problem of infrastructure network design under budget constraints: which roads to build, given a fixed budget, to maximise the number of pairs of nodes that can reach each other. Five approaches were implemented and compared across two experimental settings: an adversarially constructed synthetic graph and the real road network of Malleshwaram, Bengaluru.

The central finding is that greedy is good enough, but only the right greedy. Greedy 3, which selects edges by maximising connectivity gain per unit cost ($\rho = \Delta f / c(e)$), matches the exact DP optimum on the synthetic graph at every budget level tested, while running approximately 39 times faster. On the Malleshwaram network, where DP would require an estimated 74+ hours, Greedy 3 outperforms the other two greedy algorithms by a factor of roughly 17 in connected pairs at a representative budget level, and this dominance is consistent across all budget fractions tested.

The adversarial synthetic graph was specifically designed to surface the failure modes of each algorithm. Greedy 1, without cycle detection, wastes budget on redundant intra-cluster edges and achieves only 30.8% of DP optimal. Greedy 2 adds cycle detection, which eliminates that failure, but then fails differently: by prioritising cost over connectivity gain, it builds two internally well-connected clusters and never selects the single edge that would unite them. Only Greedy 3's gain-per-cost ratio asks the right question at each step, namely, where does the next unit of budget create the most new connectivity?

The deeper takeaway is methodological. On problems where the objective is submodular and the constraint is a budget, the theoretically grounded gain-per-cost greedy tends to perform substantially better in practice than simpler cost-based variants, even when the theoretical worst-case approximation bound is modest. On structured real-world networks, empirical performance can be significantly better than what theory guarantees. This project provides a concrete, reproducible demonstration of that phenomenon on Bengaluru's road network.

Bibliography

[1] R. Cerulli, C. D'Ambrosio, D. Serra, and C. Sorgente, *The Budgeted Labeled Minimum Spanning Tree Problem*, Mathematics, vol. 12, no. 2, p. 230, 2024. Available: https://www.mdpi.com/2227-7390/12/2/230

[2] J. Zhou, J. Zheng, and K. He, *Effective Variable Depth Local Search for the Budgeted Maximum Connected Pairs Problem*, Computational Optimization and Applications, 2022. Available: https://link.springer.com/article/10.1007/s44196-022-00096-3

[3] D. Saharoy and S. Sen, *Approximation Algorithms for Budget Constrained Network Upgradeable Problems*, 2014. Available: https://arxiv.org/pdf/1412.3721